

Multi-Level Feedback Queue on xv6-RISC-V

Week 1: Setup and Design

Aadesh Panjwani and M. Rayyan Khan

Scope

This Week 1 design document summarizes the changes introduced for adding Multi-Level Feedback Queue (MLFQ) scaffolding to xv6, including the new `getprocinfo` syscall, process metadata fields, queue structure scaffolding, and tools to observe process behavior.

What was implemented

- **User syscall:** `getprocinfo(pid, struct procinfo *buf)` exported as `SYS_getprocinfo`. Kernel code gathers process state into a `procinfo` snapshot and copies it to user space.
- **User tool:** `user/procinfo.c`, a CLI utility that prints process state, base priority, queue level, time slice budget, and per-queue runtime.
- **Proc bookkeeping fields:** `base_priority`, `queue_level`, `time_slice_budget`, `total_runtime`, `queue_runtime[]`, and `mlfq_next` added to `struct proc`.
- **MLFQ scaffolding:** Introduced in `kernel/mlfq.h` and `kernel/proc.c`, including queue structs, initialization, enqueue/dequeue helpers, and reset logic. No behavior change yet.
- **Build integration:** Makefile updated to include `procinfo` in the user program list.

Key design choices

Levels & Time Quanta

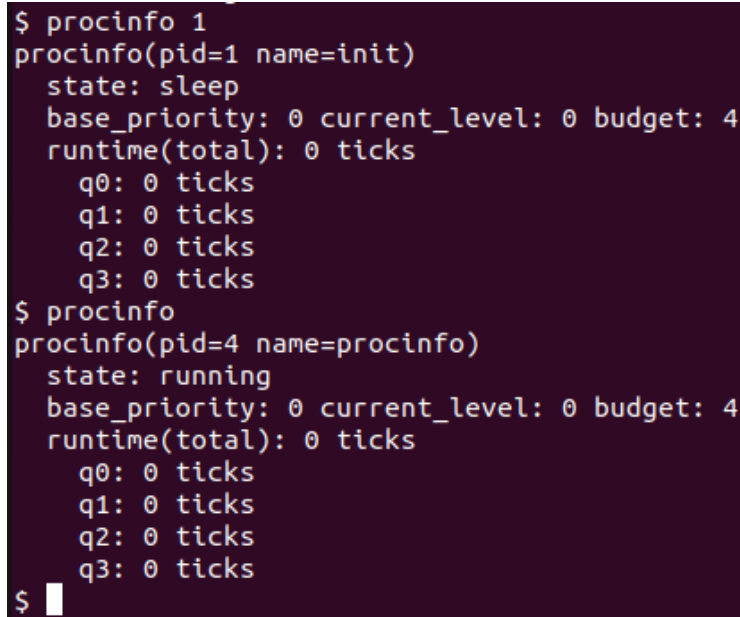
Property	Symbol	Value
Number of Levels	MLFQ_LEVELS	4 (0 = highest priority)
Time Quanta	–	{4, 8, 16, 32} ticks
Default Level	MLFQ_DEFAULT_LEVEL	0
Boost Interval (planned)	–	200 ticks

This configuration doubles quantum sizes per level, optimizing for fewer context switches for CPU-bound tasks while keeping interactive tasks responsive.

Promotion, Demotion and Yield Semantics

- **Demotion:** A process running in a queue will consume its assigned time quantum. If a process utilizes its entire time quantum without yielding (indicating a CPU-bound nature), it will be demoted to the next lower priority queue.
- **Promotion Policy (I/O-Bound Processes):** When a process performs an I/O operation and yields the CPU (e.g., enters a SLEEPING state), upon waking up and becoming RUNNABLE, it will initially retain its current priority level.
- **Promotion Policy (Starvation Prevention):** A global priority boosting mechanism will be implemented. After a predefined, large interval of system ticks, all processes will be forcibly moved back to the highest priority queue (Queue 0) to ensure fairness and prevent indefinite starvation of low-priority tasks.
- **Queue Structure:** Per-level FIFO queues using `mlfq_next` pointers; operations centralized in `proc.c`.

The following screenshot demonstrates the successful execution and output of the `procinfo` user program, showing the process's PID, state, initial priority, time slices used, and other important information for debugging.

A screenshot of a terminal window with a dark background and light-colored text. The terminal shows the execution of the 'procinfo' program twice. The first command is '\$ procinfo 1', which outputs details for PID 1 (name=init), showing it is in a 'sleep' state with a base priority of 0, current level of 0, and a budget of 4. The second command is '\$ procinfo', which outputs details for PID 4 (name=procinfo), showing it is in a 'running' state with the same priority and budget. Both outputs include runtime statistics for four time slices (q0 through q3), all currently at 0 ticks. The prompt '\$' is visible at the bottom left.

```
$ procinfo 1
procinfo(pid=1 name=init)
state: sleep
base_priority: 0 current_level: 0 budget: 4
runtime(total): 0 ticks
  q0: 0 ticks
  q1: 0 ticks
  q2: 0 ticks
  q3: 0 ticks
$ procinfo
procinfo(pid=4 name=procinfo)
state: running
base_priority: 0 current_level: 0 budget: 4
runtime(total): 0 ticks
  q0: 0 ticks
  q1: 0 ticks
  q2: 0 ticks
  q3: 0 ticks
$
```

Example output of the `procinfo` user program.

Queue Scaffolding Review

The global MLFQ state is defined in `proc.c` as:

- an array of `procqueue` structures, one per level, and
- a globally initialized `mlfq_state` containing all queues.

Each `procqueue` maintains:

- `head` – pointer to the first runnable process at the level,
- `tail` – pointer to the last process,
- FIFO ordering enforced by per-process `mlfq_next` pointers.

During system initialization, `mlfq_init()` sets all queue heads and tails to `NULL`, ensuring that the scheduler starts with a clean and deterministic queue state. Newly created processes are initialized via `mlfq_reset_proc()`, assigning:

- default priority level (0),
- initial time slice budget (based on level 0 quantum),
- zeroed runtime counters.

This guarantees consistent MLFQ metadata for all processes from their first scheduling event.

The helper functions `mlfq_enqueue()` and `mlfq_dequeue()` abstract queue manipulation:

- Enqueue operations append a process to the tail of the designated level.
- Dequeue operations remove and return the head of the level.
- Both functions preserve FIFO behavior and require no traversal.

Though unused by the scheduler during Week 1, these operations are fully functional and will be called in Week 2 when queue-aware scheduling is integrated.